

GitHub Actions

Comprehensive Technical Study Notes
Expert Reference Guide — CI/CD, Runners, Custom Actions, Security & More

1. Introduction & Core Concepts	2. Workflow File Structure
3. Triggers & Events	4. Runners
5. Contexts & Expressions	6. Job Control & Matrix
7. Secrets & Security	8. Artifacts & Caching
9. Building Custom Actions	10. Reusable Workflows
11. REST API Integration	12. Branch Protections
13. Marketplace Publishing	14. Real-World Patterns
15. Debugging & Troubleshooting	16. Summary Checklist

01 Introduction & Core Concepts

GitHub Actions is a first-class CI/CD and general-purpose automation platform built directly into GitHub. It lets you automate virtually any software workflow—build, test, deploy, notify, label, release—triggered by events in your repository or from the outside world.

Key Terms

Term	Definition
Workflow	A configurable automated process defined in a YAML file stored in <code>.github/workflows/</code> . A repo can have many workflows.
Workflow Run	A single execution instance of a workflow, spawned by a trigger event. Shows job/step progress, real-time logs, and status.
Job	A named collection of steps that runs on a single runner. Jobs run in parallel by default.
Step	An individual task within a job—either a shell command (run) or an Action (uses).
Action	A reusable, packaged unit of automation. Can be JavaScript, Docker, or composite.
Runner	The virtual or physical machine that executes a job. Can be GitHub-hosted or self-hosted.
Event	The activity that triggers a workflow (e.g., push, pull_request, a cron schedule).
Artifact	Files produced during a workflow run that can be persisted, shared between jobs, or downloaded.
Secret	An encrypted environment variable stored at the repo, environment, or org level.
Context	Objects that expose runtime data: github, env, secrets, steps, runner, job, matrix.

CI/CD Goals

- **Continuous Integration (CI):** Automatically build and test every code change, catching bugs early and ensuring the main branch is always in a deployable state.
- **Continuous Deployment (CD):** Automatically deploy verified code to staging or production environments, reducing human error and release friction.

Note: Best practice: start new workflows with simple bash echo commands to validate your runner environment before adding complex logic.

02 Workflow File Structure

All workflows live in `.github/workflows/*.yaml` (or `.yml`). GitHub scans this directory automatically.

```
name: CI Pipeline # Human-readable workflow name

on: # Trigger(s)

push:

branches: [main, develop]

pull_request:

branches: [main]

env: # Workflow-level environment variables

NODE_VERSION: '18'

jobs:

  build: # Job ID (unique within workflow)

  name: Build and Test # Human-readable job name

  runs-on: ubuntu-latest # Runner label

  steps:

    - name: Checkout code

      uses: actions/checkout@v4

    - name: Set up Node.js

      uses: actions/setup-node@v4

      with:

        node-version: ${{ env.NODE_VERSION }}

    - name: Install dependencies

      run: npm ci

    - name: Run tests

      run: npm test
```

YAML Key Reference

Key	Scope	Purpose
-----	-------	---------

name	Workflow / Job / Step	Display name in the UI
on	Workflow	Event triggers
env	Workflow / Job / Step	Environment variables (each scope narrows)
jobs	Workflow	Container for all jobs
runs-on	Job	Target runner
needs	Job	Declare job dependencies (sequential execution)
if	Job / Step	Conditional execution using expressions
uses	Step	Reference an Action by path or repo
run	Step	Execute a shell command
with	Step	Pass inputs to an Action
outputs	Job	Expose values to dependent jobs
strategy.matrix	Job	Fan-out across value combinations
timeout-minutes	Job / Step	Kill the job/step if duration exceeded
continue-on-error	Step	Allow step failure without failing the job

03 Triggers (Events)

3.1 GitHub Webhook Events

These fire on repository activity. Specify one or multiple:

```
on:

push: # Commits pushed

pull_request: # PR opened/updated

release: # Release created/published

issues: # Issue opened/closed/etc.

issue_comment: # Comment on issue or PR

workflow_dispatch: # Manual trigger (adds 'Run workflow' button in UI)

workflow_call: # Called by another workflow (reusable workflows)

repository_dispatch: # External HTTP POST trigger

deployment: # Deployment created

check_run: # Check run created/completed

milestone: # Milestone created/closed

page_build: # GitHub Pages build triggered

delete: # Branch or tag deleted
```

3.2 Trigger Filters

Narrow down which branches, tags, or file paths activate the workflow:

```
on:

push:

branches:

- main

- 'release/**' # Glob patterns supported

branches-ignore:

- 'wip/**'

tags:

- 'v*.*.*' # SemVer tags

paths:
```

```
- 'src/**' # Only when src/ changes

paths-ignore:

- '**.md' # Ignore docs-only changes
```

3.3 Scheduled Triggers (Cron)

Uses POSIX cron syntax. Runs are on UTC time. Multiple schedules can be defined.

```
on:

schedule:

- cron: '0 6 * * 1-5' # 6 AM UTC, Mon-Fri

- cron: '0 0 * * 0' # Midnight Sunday
```

Field	Range	Special Characters
Minute	0-59	, - * /
Hour	0-23	, - * /
Day of Month	1-31	, - * ? /
Month	1-12	, - * /
Day of Week	0-6 (Sun=0)	, - * /

3.4 Manual Trigger with Inputs (workflow_dispatch)

```
on:

workflow_dispatch:

inputs:

environment:

description: 'Deploy target'

required: true

default: 'staging'

type: choice

options: [staging, production]

run_tests:

description: 'Run test suite?'

type: boolean

default: true

# Access: ${ github.event.inputs.environment }
```

3.5 External Trigger (repository_dispatch)

```
on:
  repository_dispatch:
    types: [deploy-request, nightly-build] # Filter by type

# Fire from outside via curl:
# curl -X POST \
# -H 'Authorization: Bearer $TOKEN' \
# https://api.github.com/repos/OWNER/REPO/dispatches \
# -d '{"event_type": "deploy-request", "client_payload": {"env": "prod"}}'

# Access payload: ${ github.event.client_payload.env }
```

3.6 Pull Request Activity Types

Category	Types
Lifecycle	opened, synchronize, reopened, edited, closed
Reviews	review_requested, review_request_removed, ready_for_review
Management	assigned, unassigned, labeled, unlabeled, locked, unlocked

04 Runners

4.1 GitHub-Hosted Runners

Label	OS	Notes
ubuntu-latest / ubuntu-24.04	Ubuntu 24.04	Most common; fastest startup; recommended default
ubuntu-22.04	Ubuntu 22.04	LTS, very stable
ubuntu-20.04	Ubuntu 20.04	Older LTS, legacy support
windows-latest / windows-2022	Windows Server 2022	Full Windows toolchain
windows-2019	Windows Server 2019	Older Windows builds
macos-latest / macos-14	macOS 14 Sonoma	ARM (Apple M1); slower startup than Linux
macos-13	macOS 13 Ventura	x64 Intel
macos-12	macOS 12 Monterey	x64 Intel, older

Note: GitHub-hosted runners come pre-installed with Docker, Python, Node.js, Java, Ruby, Go, .NET, Terraform, kubectl, Helm, AWS CLI, Azure CLI, and more.

4.2 Self-Hosted Runners

Useful for: custom hardware, GPU workloads, on-premises security requirements, or cost control for large workloads.

```
# Target self-hosted runners by label:  
  
runs-on: self-hosted # Any self-hosted runner  
  
runs-on: [self-hosted, linux, arm64] # With OS + arch labels  
  
runs-on: [self-hosted, gpu] # Custom label
```

OS	Requirements
Linux	x64, ARM64, ARM32 (ARM32 requires ARM64 hardware base)
Windows	Windows 7/8.1/10, Server 2012 R2/2016/2019/2022 — ALL must be 64-bit
macOS	macOS 10.13+

- **Ephemeral runners (recommended):** Use `--ephemeral` flag so runners deregister after one job, preventing state leakage between runs.
- **Runner Groups:** Org-level feature to restrict which repos can use which self-hosted runners.

05 Contexts and Expressions

5.1 Context Objects

Context	Key Properties
github	.event, .sha, .ref, .repository, .actor, .workflow, .run_id, .run_number, .event_name
env	All environment variables in scope
secrets	GITHUB_TOKEN, and all custom secrets you've defined
steps	.outputs, .conclusion, .outcome of prior named steps
job	.status of the current job
runner	.os, .arch, .temp, .tool_cache
matrix	Current matrix dimension values (e.g., matrix.node, matrix.os)
needs	.outputs, .result from jobs listed in needs
inputs	Values from workflow_dispatch or workflow_call inputs

5.2 Expression Syntax & Functions

```
${{ expression }}
```

Operators: == != < > <= >= && || !

String functions:

```
contains(string, substring) # true/false
startsWith(string, prefix)
endsWith(string, suffix)
format('{0} by {1}', github.sha, github.actor)
join(array, separator)
```

Type conversion:

```
toJSON(value) # Serialize to JSON string
fromJSON(string) # Parse JSON string
hashFiles('**/package-lock.json') # SHA-256 hash of matched files
```

Job/Step status functions:

```
success() # true if all prior steps passed
failure() # true if any prior step failed
```

```
always() # always runs (even on cancel)

cancelled() # true if workflow was cancelled

# Common conditional examples:

if: github.ref == 'refs/heads/main'

if: github.event_name == 'pull_request'

if: failure()

if: always()

if: contains(github.event.pull_request.labels.*.name, 'deploy')
```

06 Job Control

6.1 Sequential Jobs with 'needs'

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps: [...]

  test:
    needs: build # Waits for build to complete
    runs-on: ubuntu-latest
    steps: [...]

  deploy:
    needs: [build, test] # Waits for BOTH
    runs-on: ubuntu-latest
    steps: [...]
```

6.2 Passing Data Between Jobs

Jobs run on isolated runners, so data must be shared explicitly via outputs:

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${{ steps.get_version.outputs.version }}
    steps:
      - id: get_version
        run: echo "version=$(cat VERSION)" >> $GITHUB_OUTPUT

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
```

```
- run: echo "Deploying version ${ needs.build.outputs.version }"
```

6.3 Matrix Strategy

Fan-out a job across multiple configurations simultaneously:

```
jobs:
  test:
    strategy:
      fail-fast: false # Don't cancel all if one fails
      max-parallel: 4
    matrix:
      os: [ubuntu-latest, windows-latest, macos-latest]
      node: [16, 18, 20]
      exclude:
        - os: windows-latest
          node: 16
      include:
        - os: ubuntu-latest
          node: 20
      experimental: true
      runs-on: ${ matrix.os }
    steps:
      - uses: actions/setup-node@v4
    with:
      node-version: ${ matrix.node }
```

6.4 Concurrency Control

Prevent parallel runs on the same branch to avoid race conditions:

```
concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: true # Cancel older run when new one starts
```

6.5 Job Environments (Deployment Protection)

```
jobs:  
  
  deploy:  
  
    environment:  
  
      name: production  
  
      url: https://myapp.com # Shown in deployment UI  
  
      runs-on: ubuntu-latest
```

Environment features: required reviewers, wait timers, environment-scoped secrets, and full deployment history.

07 Secrets and Security

7.1 GITHUB_TOKEN

Automatically injected into every workflow run. Scoped to the current repository. Access via `${{ secrets.GITHUB_TOKEN }}`.

```
steps:
  - name: Call GitHub API
    run: |
      curl -X POST \
        -H "Authorization: Bearer ${{ secrets.GITHUB_TOKEN }}" \
        -H "Accept: application/vnd.github+json" \
        https://api.github.com/repos/${{ github.repository }}/issues \
        -d '{"title": "Auto-issue for ${{ github.sha }}"}'
```

Restrict permissions per workflow (least-privilege):

```
permissions: # Workflow-level
  contents: read
  issues: write
  pull-requests: write

jobs:
  build:
    permissions: # Job-level (narrower scope wins)
      contents: read
```

7.2 Repository / Org / Environment Secrets

```
steps:
  - run: deploy.sh

env:
  API_KEY: ${{ secrets.MY_API_KEY }}
  DB_PASS: ${{ secrets.DB_PASSWORD }}
```

- Secrets are **masked in logs automatically** — they never appear in plain text.
- Forked repos do **NOT** have access to parent repo secrets by default (security boundary).
- Use **Environment secrets** to scope credentials to specific deployment environments.

7.3 OpenID Connect (OIDC) — Keyless Cloud Auth

Instead of storing long-lived cloud credentials as secrets, use OIDC to get short-lived tokens minted per-run. Supported by AWS, Azure, GCP, HashiCorp Vault, and others.

```
permissions:

  id-token: write

  contents: read

steps:

  - uses: aws-actions/configure-aws-credentials@v4

  with:

    role-to-assume: arn:aws:iam::123456789:role/GitHubActionsRole

    aws-region: us-east-1

    # No AWS secrets stored anywhere – credentials are minted per-run!
```

08 Artifacts and Caching

8.1 Upload Artifacts

```
- name: Upload test results

uses: actions/upload-artifact@v4

with:
  name: test-results
  path: |
    test-output/
    coverage/

  retention-days: 14 # Default is 90

  if-no-files-found: error # warn | ignore | error
```

8.2 Download Artifacts (Between Jobs)

```
- name: Download test results

uses: actions/download-artifact@v4

with:
  name: test-results
  path: ./downloaded-results
```

8.3 Caching Dependencies

Artifacts != Cache. The cache action stores and restores dependency directories between runs, dramatically speeding up workflows.

```
- uses: actions/cache@v4

with:
  path: ~/.npm

  key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }

  restore-keys: |
    ${ runner.os }-node-

# Other common cache paths:
# pip: ~/.cache/pip
```



```
# Maven: ~/.m2  
  
# Gradle: ~/.gradle/caches  
  
# Cargo: ~/.cargo/registry
```

Note: Keys use `hashFiles()` to auto-invalidate when lock files change. `restore-keys` provides fallback cache hits.

09 Building Custom Actions

9.1 Types of Actions

Type	Language	Runtime	Best For
JavaScript	JS / TypeScript	Node.js 20	Fast startup, complex logic, GitHub API calls
Docker Container	Any language	Docker	Custom environment, any language/runtime
Composite	YAML	Runner shell	Combining existing steps/actions without code

9.2 Action Metadata (action.yml)

Every action requires an **action.yml** file at its repo root:

```
name: 'My Custom Action'

description: 'Does something awesome'

author: 'Your Name'

branding:

  icon: 'zap' # Any Feather icon name

  color: 'yellow' # red, orange, yellow, green, blue, purple, gray-dark

inputs:

  api-url:

    description: 'The API endpoint to call'

    required: true

    timeout:

      description: 'Request timeout in seconds'

      required: false

      default: '30'

  outputs:

    response-code:

      description: 'HTTP response code from the API'

    response-body:

      description: 'Response body as a string'

runs:
```

```
using: 'node20' # node20 | docker | composite

main: 'dist/index.js' # Entry point

post: 'dist/cleanup.js' # Optional: runs after job completes
```

9.3 JavaScript Action

Directory structure:

```
my-action/
├── action.yml
├── src/
│   └── index.js
├── dist/
│   └── index.js # Compiled/bundled – always commit this!
├── package.json
└── README.md
```

src/index.js:

```
const core = require('@actions/core');
const github = require('@actions/github');

async function run() {
  try {
    const apiUrl = core.getInput('api-url', { required: true });
    const context = github.context;

    core.info(`Running on: ${context.repo.owner}/${context.repo.repo}`);

    // ... do work ...

    core.setOutput('response-code', '200');

    // Add a rich job summary:
    await core.summary
      .addHeading('Action Results')
      .addTable([[{data: 'Field', header: true}, {data: 'Value', header: true}],
        ['Status', 'Success']))
```

```
.write();

} catch (error) {

  core.setFailed(`Action failed: ${error.message}`);

}

}

run();
```

Key @actions/core APIs:

Method	Purpose
core.getInput(name)	Read an input value (string)
core.getBooleanInput(name)	Read input as boolean
core.setOutput(name, value)	Set an output value for downstream steps/jobs
core.setFailed(message)	Fail the step and job with a message
core.info/warning/error/debug(msg)	Log at various levels
core.setSecret(value)	Mask a value in all subsequent log output
core.exportVariable(name, val)	Set env var for subsequent steps
core.addPath(path)	Prepend directory to PATH
core.group(name, fn)	Collapsible log group for noisy output
core.saveState / getState	Persist data between main and post step

Build and bundle:

```
npm install @actions/core @actions/github @actions/http-client

npm install --save-dev @vercel/ncc

npx ncc build src/index.js -o dist --minify

# Always commit dist/ so consumers don't need npm install
```

9.4 Composite Action

Combine multiple steps into one reusable action — no compilation needed:

```
runs:

  using: 'composite'

steps:

  - name: Setup Node
```

```
uses: actions/setup-node@v4

with:
  node-version: ${{ inputs.node-version }}

- name: Install

  run: npm ci

  shell: bash # shell is REQUIRED in composite steps

- name: Build

  id: build

  run: |

    npm run build

  echo "path=dist/" >> $GITHUB_OUTPUT

  shell: bash
```

9.5 Modern Output Syntax (REQUIRED)

The old `::set-output::` syntax is deprecated. Always use the file-based approach:

```
# Set step outputs:

echo "my-output=hello" >> $GITHUB_OUTPUT

# Multiline output:

echo "my-output<<EOF" >> $GITHUB_OUTPUT

echo "line1" >> $GITHUB_OUTPUT

echo "line2" >> $GITHUB_OUTPUT

echo "EOF" >> $GITHUB_OUTPUT

# Set env vars for subsequent steps:

echo "MY_VAR=hello" >> $GITHUB_ENV

# Extend PATH:

echo "/my/tool/bin" >> $GITHUB_PATH

# Rich job summary (rendered as Markdown in the Actions UI):

echo "## Test Results" >> $GITHUB_STEP_SUMMARY

echo "| Suite | Status |" >> $GITHUB_STEP_SUMMARY
```

```
echo "|-----|-----|" >> $GITHUB_STEP_SUMMARY  
  
echo "| Unit | Pass |" >> $GITHUB_STEP_SUMMARY
```

10 Reusable Workflows

Factor out entire workflows to share across repos. The called workflow must use **workflow_call** as its trigger.

Caller Workflow

```
jobs:

  call-build:

    uses: myorg/shared-workflows/.github/workflows/build.yml@main

    with:

      node-version: '20'

    secrets: inherit # Pass all secrets down automatically
```

Callee Workflow (.github/workflows/build.yml)

```
on:

  workflow_call:

  inputs:

    node-version:

      required: true

      type: string

    secrets:

      NPM_TOKEN:

        required: false

    outputs:

      artifact-name:

        description: 'Name of the uploaded artifact'

        value: ${ jobs.build.outputs.artifact-name }

  jobs:

    build:

      runs-on: ubuntu-latest

      outputs:

        artifact-name: ${ steps.upload.outputs.artifact-id }

      steps:
```

```
- uses: actions/checkout@v4  
  
- uses: actions/setup-node@v4  
  
with:  
  
node-version: ${{ inputs.node-version }}
```


11 REST API Integration

11.1 Creating Issues via REST API

```
- name: Create issue

run: |
  curl -X POST \
    -H "Authorization: Bearer ${ secrets.GITHUB_TOKEN }" \
    -H "Accept: application/vnd.github+json" \
    -H "X-GitHub-Api-Version: 2022-11-28" \
    https://api.github.com/repos/${ github.repository }/issues \
    -d '{
      "title": "Build failed for ${ github.sha }",
      "body": "Workflow: ${ github.workflow }\\nRun: ${ github.run_id }",
      "labels": ["bug", "automated"]
    }'
```

11.2 Using the gh CLI (Pre-installed on All GitHub-Hosted Runners)

```
- name: Create PR

env:
  GH_TOKEN: ${ secrets.GITHUB_TOKEN }

run: |
  gh pr create \
    --title "Auto PR: ${ github.ref_name }" \
    --body "Automated by GitHub Actions" \
    --base main \
    --head ${ github.ref_name }

- name: Comment on PR

env:
  GH_TOKEN: ${ secrets.GITHUB_TOKEN }

run: |
  gh pr comment ${ github.event.pull_request.number } \
```

```
--body "Tests passed!"

- name: Create release
  env:
    GH_TOKEN: ${ secrets.GITHUB_TOKEN }
  run: |
    gh release create v${ github.run_number } \
      --title "Release v${ github.run_number }" \
      --notes "Automated release" \
      dist/*.tar.gz
```

12 Branch Protections & Code Quality Gates

12.1 Branch Protection Rules

Configure in Repo → Settings → Branches → Add rule:

- **Require status checks to pass** before merging — tie your CI job names here
- **Require branches to be up to date** before merging
- **Require pull request reviews** (N approvals required)
- **Dismiss stale reviews** when new commits are pushed to the PR
- **Require review from Code Owners** (via CODEOWNERS file)
- **Restrict who can push** directly to the branch
- **Require signed commits** (GPG or SSH signature verification)

12.2 CODEOWNERS

```
# .github/CODEOWNERS

* @org/team-core # Default owner for all files

src/api/** @org/team-backend

src/ui/** @org/team-frontend

*.yml @org/team-devops

.github/** @org/team-devops
```

12.3 PR Approval Labeling

```
- uses: abinoda/label-when-approved-action@1.0.5

with:

  approved-label: 'approved'

env:

  GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

13 Marketplace Publishing Checklist

Step	Requirement
1. Repository	Action repo must be public. Root action.yml must exist.
2. Metadata	Complete name, description, author, branding (icon + color), all inputs/outputs documented.
3. README.md	Usage examples, inputs/outputs table, requirements, license.
4. Semantic Versioning	Tag releases: v1.0.0. Also maintain floating major tag (v1) pointing to latest stable.
5. Branch Protections	Enable on default branch of your action repo.
6. Approval Labeling	Add automated PR labeling for code review visibility.
7. Publish	Create a new GitHub Release — GitHub will offer 'Publish to Marketplace'.

Maintaining Floating Version Tags

```
# After releasing v1.2.3, update the floating v1 tag:

git tag -fa v1 -m "Update v1 tag to v1.2.3"

git push origin v1 --force

# Users can pin to: actions/my-action@v1 (gets latest v1.x.x)

# or: actions/my-action@v1.2.3 (exact version)
```

14 Real-World Workflow Patterns

14.1 Full CI Pipeline (Lint → Test → Build)

```
name: CI

on:
  push:
  branches: [main]
  pull_request:
  branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with: { node-version: '20', cache: 'npm' }
      - run: npm ci && npm run lint

  test:
    needs: lint
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node: [18, 20]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with: { node-version: ${ matrix.node }, cache: 'npm' }
      - run: npm ci && npm test -- --coverage
      - uses: actions/upload-artifact@v4
    with:
```

```
name: coverage-${{ matrix.node }}

path: coverage/

build:

needs: test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: actions/setup-node@v4

with: { node-version: '20', cache: 'npm' }

- run: npm ci && npm run build

- uses: actions/upload-artifact@v4

with:

name: dist

path: dist/
```

14.2 Deploy on Merge to Main (OIDC to AWS)

```
name: Deploy

on:

push:

branches: [main]

jobs:

deploy:

runs-on: ubuntu-latest

environment:

name: production

url: https://myapp.com

permissions:

id-token: write

contents: read

steps:

- uses: actions/checkout@v4
```

```
- uses: aws-actions/configure-aws-credentials@v4

with:

  role-to-assume: ${ secrets.AWS_ROLE_ARN }

  aws-region: us-east-1

- run: aws s3 sync dist/ s3://my-bucket --delete
```

15 Debugging & Troubleshooting

15.1 Enable Debug Logging

Set these as repository secrets with value **true** for verbose output:

Secret Name	Effect
ACTIONS_STEP_DEBUG	Shows debug-level logs from steps (<code>core.debug()</code> calls become visible)
ACTIONS_RUNNER_DEBUG	Shows runner diagnostic logs (machine-level detail)

15.2 Dump Context for Inspection

```
- name: Dump GitHub context
run: echo '${{ toJSON(github) }}'

- name: Dump all contexts
run: |
echo "github: ${{ toJSON(github) }}"
echo "env: ${{ toJSON(env) }}"
echo "job: ${{ toJSON(job) }}"
echo "runner: ${{ toJSON(runner) }}"
```

15.3 Interactive SSH Debugging with tmate

```
- name: Setup tmate session (SSH into the runner)

if: failure() # Only on failure

uses: mxschmitt/action-tmate@v3

with:

  limit-access-to-actor: true # Only the triggering user can connect
```

Note: tmate opens an interactive SSH session into the live runner — invaluable for debugging environment issues.

15.4 Other Debugging Tips

- **Re-run failed jobs:** In the Actions UI, 'Re-run failed jobs' only reruns failed jobs, preserving successful ones. Good for flaky tests.
- **act (local runner):** The open-source 'act' tool lets you run GitHub Actions locally using Docker. Great for rapid iteration.
- **Annotations:** `core.error()` and `core.warning()` create inline annotations on your PR diff — use them to surface actionable errors.

- **Job summaries:** Write Markdown to `$GITHUB_STEP_SUMMARY` to create rich reports visible in the workflow run UI.

16 Summary Checklist

[]	Trigger Precision	Use branches, paths, tags filters to limit unnecessary runs
[]	Baseline Validation	Verify environment health with simple bash commands before full automation
[]	Log Mastery	Navigate workflow logs; enable ACTIONS_STEP_DEBUG for verbose output
[]	Credential Security	Use <code>\${{ secrets.GITHUB_TOKEN }}</code> and named secrets; prefer OIDC over stored keys
[]	Least Privilege	Scope permissions per workflow or job to the minimum required
[]	Runner Optimization	Match runs-on to your OS; use self-hosted for custom hardware/GPU
[]	Job Dependencies	Use needs for sequential stages; outputs to pass data between jobs
[]	Matrix Builds	Test across OS/runtime combinations with <code>strategy.matrix</code>
[]	Artifact Persistence	Upload build outputs and test reports; use cache for dependencies
[]	Custom Actions	JS for speed/API, Docker for custom runtimes, Composite for YAML reuse
[]	Modern Output Syntax	Use <code>\$GITHUB_OUTPUT</code> , <code>\$GITHUB_ENV</code> , <code>\$GITHUB_STEP_SUMMARY</code> (not deprecated <code>::set-output::</code>)
[]	Reusable Workflows	Extract common pipelines with <code>workflow_call</code> for DRY cross-repo sharing
[]	Branch Protections	Require status checks, reviews, and signed commits on protected branches
[]	Marketplace Publishing	Complete <code>action.yml</code> metadata, README, versioned tags, and branding
[]	Concurrency Control	Prevent duplicate runs with concurrency groups and cancel-in-progress
[]	Debug Mode	Toggle <code>ACTIONS_STEP_DEBUG</code> and <code>ACTIONS_RUNNER_DEBUG</code> secrets for verbose logs

Curated Actions Reference

Action	Version	Purpose
--------	---------	---------

actions/checkout	@v4	Clone the repository into the runner
actions/setup-node	@v4	Install and configure Node.js
actions/setup-python	@v5	Install and configure Python
actions/cache	@v4	Cache dependencies between runs
actions/upload-artifact	@v4	Persist build outputs
actions/download-artifact	@v4	Retrieve stored build outputs
aws-actions/configure-aws-credentials	@v4	OIDC auth to AWS
azure/login	@v2	OIDC auth to Azure
google-github-actions/auth	@v2	OIDC auth to GCP
docker/login-action	@v3	Authenticate to Docker Hub / GHCR
docker/build-push-action	@v5	Build and push Docker images
abinoda/label-when-approved-action	@1.0.5	Automated PR labeling on approval
mxschmitt/action-tmate	@v3	Interactive SSH debugging session